

PlaceMate: A Full-Stack Placement Cell Management System with Server-Side Policy Enforcement

Tushar Kumar

Department of Computer Applications (BCA)

Maharaja Surajmal Institute, C-4 Janakpuri, New Delhi – 110058, India

(Affiliated to Guru Gobind Singh Indraprastha University, Delhi)

September 2026

Abstract—During placement season a college must match several hundred final-year students against dozens of recruitment drives under hard eligibility rules, and at most institutes this still happens in spreadsheets edited by a small office staff. This paper presents PlaceMate, a full-stack placement cell management system built on a Flask and PostgreSQL REST API with a React single-page frontend. The design concentrates every application rule in one server-side eligibility function, adds an offer-ladder policy backed by an automatically maintained placement-record table, and keeps an append-only status history for every application so that decisions remain auditable. Because the production deployment splits the interface and the API across two subdomains, the session model and a request-origin guard for cross-site request forgery are treated as first-class design concerns. A scripted integration suite of 23 REST-level assertions verifies the full workflow on a scratch database. A performance pass reduced the initial JavaScript payload from 740 kB to 336 kB (206 kB to 91 kB compressed) by splitting the dashboard bundle, and reduced per-request database queries by replacing per-row counts and lazy loads with joined and grouped queries.

Index Terms—campus placement, web application, Flask, React, PostgreSQL, eligibility rules, audit trail, performance engineering.

I. Introduction

Every autumn the placement cell of a college turns into a small operations centre. Several hundred final-year students, a few dozen visiting companies and a rolling calendar of recruitment drives have to be matched against each other under rules that do not bend: a minimum CGPA here, a department restriction there, a deadline that closes a drive overnight. At most institutes this matching still runs through spreadsheets that a handful of staff members edit in turns. The sheets drift out of date. Nobody can say afterwards why a particular student was rejected. Students hear about drives days late and apply by email, which the cell then re-enters by hand.

PlaceMate is a web application built to replace that workflow. It serves two roles. The placement cell (admin) maintains students, companies, drives and notices, records application decisions and reads a live dashboard. Students browse the drives they are actually eligible for, apply in one click, track each round of their applications and keep their own contact details current. The system is a conventional REST service [4] with a JSON API written in Flask [5] over PostgreSQL, consumed by a React single-page application.

The work makes four contributions. First, all eligibility logic lives in a single server-side function through which every application decision passes, so the rules cannot drift between the student interface and the admin interface. Second, an offer-ladder policy backed by an automatically maintained placement-record table prevents a student who already holds an offer from sitting for a drive that pays less than their best offer plus a fixed increment. Third, every application transition is written to an append-only status-history table, which gives both roles a per-application timeline and gives the cell an audit trail. Fourth, the client

was measured and restructured, cutting the initial JavaScript payload by more than half while keeping the interface unchanged.

The rest of the paper is organised as follows. Section II reviews related placement systems. Section III presents the design of the eligibility engine, the offer policy and the audit trail. Section IV covers implementation details, including session handling and the performance work. Section V reports functional and performance evaluation, Section VI states limitations, and Section VII concludes.

II. Related Work

Web-based placement systems have been described repeatedly in the recent literature. Kousik proposes a MERN-stack application that acts as a placement management interface for institutions [1]. Prakash describes an integrated platform, Placement Connect, which adds analysis of historical placement data to identify process bottlenecks [2]. Priyanka presents CART, a system built around the placement-cell concept for managing student and company records [3]. These systems settle the information-management problem: records, listings and status fields are all covered.

What receives less attention is where the rules live. In several of these designs, eligibility checking is a concern of the user interface, which means the underlying API will accept whatever the form allows and the rules are only as good as the last page that remembered to validate. PlaceMate takes the opposite position: the interface may render whatever it likes, but the API alone decides. The second gap is auditability. None of the surveyed systems describe an application-level history that records who moved a candidate to which stage and when; in PlaceMate this is a first-class table. On the engineering side the system follows REST practice [4], uses Flask on the server [5] and pandas [6] for the department-level aggregation behind the dashboard, deploys on a serverless PostgreSQL service whose storage-compute split is described by Vanlightly [7], and follows the OWASP session-management and CSRF-prevention guidance [8], [9].

III. System Design

A. Roles and Data Model

The relational schema has six core tables. users holds login identities with a role field; students, companies and drives hold the domain records; applications links a student to a drive and carries the current stage (applied, shortlisted, selected or rejected); notices holds announcements. A unique constraint on (student, drive) backs the duplicate-application rule at the database level, which also settles the race between two concurrent apply requests without application-level locking. Two tables extend this core: placement_records stores actual offers, and application_status_history stores the audit trail; both are described below.

B. The Eligibility Engine

All eligibility logic sits in one function on the Drive model that returns a pair: an allow flag and, when the answer is negative, a reason string. The function checks, in order, whether the application deadline has passed, whether the drive is closed, whether the student's programme and department are listed as eligible, and whether the student's CGPA clears the drive's minimum. Four call sites use this one function: the student's drive listing, the apply endpoint, the admin's per-application eligibility badge, and the admin's list of eligible students who have not yet applied. When a rule changes, it changes in one place, and the reason string

reaches the student directly (“Needs CGPA ≥ 7 ”), which in practice answers most queries that would otherwise arrive at the placement office.

C. Offers and the Offer Ladder

Marking an application as selected is treated as making an offer. The API then inserts a placement record holding the student, the drive, the offer date and the offered package, seeded from the drive's advertised figure. The cell can overwrite the package with the actual CTC, which often differs from the advertised number once bonuses are settled. Moving the application off selected deletes the record, so the offer table and the application states can never disagree for long. The ladder rule then follows: a student holding at least one placement record may only apply to drives whose package is at least their best offer plus 2.0 LPA. Drives without a published package are left open, since the rule cannot be evaluated against them. The dashboard's highest and average package figures read from the placement records when records exist and fall back to advertised drive figures otherwise.

D. Status History

Every status change, including the initial application itself, appends a row to application_status_history with the new stage, an optional note, the acting user and a timestamp. Nothing updates or deletes these rows except the cascade that runs when a whole application is removed. The table drives a timeline view on the admin's application detail page and a compact stage list on the student's applications page, and it doubles as the audit record of the season: the cell can reconstruct afterwards exactly when each decision was made.

E. Sessions in a Split Deployment

Authentication uses Flask's signed client-side session cookie. In production the frontend and the API deploy as separate services on different subdomains, so the cookie is issued with SameSite=None and the Secure flag, which is the configuration recommended for cross-site session cookies [8]. That choice has a consequence worth stating plainly: the browser will also attach the cookie to cross-site POSTs from any origin, and CORS cannot prevent simple form posts from being sent [9]. Section IV-C describes the guard this forces.

IV. Implementation

A. Stack

The backend is a Flask 3 application factory with three blueprints (authentication, admin, portal) over SQLAlchemy on PostgreSQL through psycopg; in deployment the database is Neon, a serverless Postgres whose compute and storage are separated [7]. Every admin and portal route sits behind a role-checking decorator, and the production configuration refuses to boot without a real secret key, so the development fallback cannot leak into a deployment. The frontend is React 19 with Vite and Tailwind CSS 4; the dashboard charts use Recharts.

B. Account Provisioning and Data Exchange

Administrators add students one at a time or import a CSV. The importer parses every column as text so that numeric roll numbers and phone numbers survive, caps a file at 5,000 rows, and reports skipped rows with reasons rather than failing the whole file. Each newly created student receives a login whose password is a randomly generated one-time value, shown to the administrator exactly once at creation or through an explicit reset action; nothing resembles a default password. The CSV export path prefixes any cell beginning with =, +, - or @ with an apostrophe, which neutralises spreadsheet formula injection when the file is opened in Excel or Sheets.

C. Request-Origin Guard

Because the production cookie is SameSite=None, the server verifies the origin of every state-changing request. Before a POST, PUT, PATCH or DELETE is dispatched, the host in the request's Origin or Referer header must match either the API's own host or the configured frontend origin. Requests without either header, which is how non-browser clients behave, are allowed; a browser-sent cross-site POST is refused with 403. The check runs only in production so that the Vite development proxy is not affected.

D. Client Performance Work

The built application initially shipped a single 740 kB JavaScript bundle (206 kB compressed) because the charting library rode along with every route. The dashboard is now loaded through a dynamic import, so the entry bundle holds only the shell and the list pages, and the 404 kB chart module arrives on the first visit to the dashboard. Table I lists the figures.

A rendering pass followed. The animated backdrop combines drifting colour blobs, a ribbed-glass overlay and a travelling highlight. Two of these were expensive in ways their appearance did not justify. The highlight animated background-position, a property that forces a full-viewport repaint every frame, and on Chromium the overlay carried a full-screen SVG-displacement backdrop filter whose displacement map ran at scale 1, shifting pixels by at most about a quarter pixel, which is invisible under the overlay's own tinting. The filter was removed and the sweep was rebuilt to animate a transform on an oversized layer, with the tile size chosen so that highlight positions and loop timing match the original. The page gradient moved from background-attachment: fixed, which Safari repaints on scroll, to a fixed pseudo-element. After the pass every animation on the page runs on transform or opacity alone.

E. Query Reduction

Three list endpoints issued per-row work. The drives page issued one COUNT query per drive plus one for the company filter list; the applications page lazily loaded the student, drive and company of every row; the student detail page behaved similarly for a student's applications and offers. Each now issues a constant number of queries: joined eager loads for the row data and a single grouped aggregate for the counts. Table II gives the per-request counts, where N is the number of rows returned.

V. Evaluation

A. Functional Verification

A scripted integration suite exercises the running application against a scratch PostgreSQL database through the full HTTP surface. Its 23 assertions cover administrator login, creating companies and drives, creating a student and retrieving the one-time password, student login with that password, the portal drive listing and detail, applying to a drive, the offer appearing when the application is marked selected, the ladder rejecting an application to a lower-paying drive with its reason, the offer disappearing when the decision is walked back, dashboard figures reflecting the overridden CTC, cascade deletion of students, drives and companies including their history and offer rows, and the origin guard returning 403 for a foreign origin while letting the configured origin and header-less clients through. The suite passed in full before the performance work and again after it, which is the evidence that the optimisations changed cost and not behaviour.

B. Payload Size

TABLE I

Initial JavaScript Payload Before and After Code Splitting

Bundle	Before	After	Change
Entry, raw	740 kB	336 kB	−55%
Entry, gzip	206 kB	91 kB	−56%
Dashboard chunk	—	404 kB	on demand
Dashboard chunk, gzip	—	114 kB	on demand

Every route other than the dashboard now paints from the smaller entry chunk. The deferred chart module is fetched once and cached by the browser afterwards.

C. Database Queries per Request

TABLE II

Query Count per Request Before and After the Rework

Endpoint	Before	After
Drives list	$N + 2$	3
Applications list	$3N + 1$	1
Student detail	$\approx 2N + 4$	4
Companies list	$N + 1$	2

The reductions matter most on the deployments this system targets, where each serverless request pays connection setup to a remote database; removing dozens of round trips per page load is worth more than micro-optimising any single query.

D. Rendering Cost

After the changes, all continuous animation on the page is compositor-owned: the drifting blobs and the travelling highlight animate transforms, the fireflies animate transform and opacity, and no CSS animation touches layout or paint properties. The rib overlay and highlight positions were kept dimensionally identical to the previous implementation, so the change is intended to be indistinguishable to a user; the verification is visual inspection rather than instrumentation, which Section VI lists among the limitations.

VI. Limitations

The list endpoints are unbounded and will need server-side pagination once a real cohort approaches the importer's 5,000-row ceiling. Login has no rate limiting, which is tolerable for a campus tool and not for a public service. The integration suite runs on demand rather than in CI, so its guarantee depends on the developer remembering to run it. The performance figures come from one development machine and one build; they measure the payload, which is objective, but the rendering discussion is qualitative. Finally, the offer ladder trusts the advertised package until the cell overrides it, so a drive listed with an inflated figure temporarily admits students the rule would otherwise exclude.

VII. Conclusion and Future Work

PlaceMate replaces the spreadsheet season with a system whose rules live in one place, whose offers are recorded as data rather than as memory, and whose decisions can be replayed from an append-only log. The engineering evaluation shows that the usual costs of a rich single-page

interface, a heavy first payload and chatty database access, could be reduced by ordinary means: code splitting, joined queries and grouped aggregates, and a rendering pass that keeps every animation on the compositor.

Planned work follows the limitations. Server-side pagination and login rate limiting come first, then resume upload attached to the student record, notifications driven by the status-history table, skill-based matching between student profiles and drive requirements, and finally a read-only recruiter view so that visiting companies can see their own shortlists.

References

- [1] R. K. Kousik, “Computer human interface for placement management,” IEEE Xplore, doc. 10486671, 2024.
- [2] D. S. Prakash, “An integrated web-based platform for enhanced college placement management and student engagement,” IEEE Xplore, doc. 10717061, 2024.
- [3] G. Priyanka, “CART – web-based institutional placement management system,” IEEE Xplore, doc. 10716877, 2024.
- [4] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Univ. of California, Irvine, CA, USA, 2000.
- [5] M. Grinberg, Flask Web Development: Developing Web Applications with Python, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [6] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf. (SciPy), Austin, TX, USA, 2010, pp. 56–61.
- [7] J. Vanlightly, “Neon: Serverless PostgreSQL,” Architecture Decision Series, ch. 3, Nov. 2023. [Online]. Available: <https://jack-vanlightly.com/analyses/2023/11/15/neon-serverless-postgresql-asds-chapter-3>
- [8] OWASP Foundation, “Session management cheat sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- [9] OWASP Foundation, “Cross-site request forgery prevention cheat sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Backmatter

Data availability: The system source, schema and test scripts are held in the author's private repository; no external datasets were used or generated.

Author contributions: Single-author work. The author designed the system, implemented it, ran the evaluation and wrote the paper (CRedit roles: conceptualisation, methodology, software, validation, investigation, writing).

Acknowledgment of tooling: AI-based coding assistants were used for implementation support, editing and document preparation. The system design, measurements, analysis and conclusions are the author's own.

Conflict of interest: The author declares no conflict of interest.

Funding: This work received no external funding.

Ethics: The system processes student placement records under institutional access control; no human-subjects data was collected for this paper.